

Analytic Generative Design of Quasi-Symmetric Stellarators: An Inverse Riemannian Embedding Approach

David

February 6, 2026

Abstract

The commercial realization of stellarator fusion energy is hindered in part by the computational cost and non-convexity of 3D equilibrium and configuration optimization workflows. Many modern pipelines perform repeated high-dimensional searches (e.g., VMEC/STELLOPT-based loops) that are expensive and sensitive to initialization. This paper presents a deterministic, analytic framework for constructing *near-axis* stellarator *core geometries* that satisfy **quasi-symmetry (QS)** constraints *to first order in a near-axis expansion (NAE)*. Using the NAE to invert the equilibrium/shaping relationships near the magnetic axis, we obtain a **Riccati ordinary differential equation** for a complex cross-section parameter σ that determines the leading-order rotating-ellipse flux-surface geometry from magnetic-axis properties. The resulting generator reduces the initial core-shaping step from a high-dimensional variational search to integration of a single ODE together with a periodicity condition, enabling rapid generation of QA-consistent near-axis candidates that can seed subsequent full-equilibrium and coil-design stages. We provide a derivation summary and a Python implementation that constructs a Quasi-Axisymmetric (QA) near-axis configuration.

1 Introduction

The stellarator offers a pathway to steady-state nuclear fusion, free from the current-driven disruptions of the tokamak. However, a generic three-dimensional toroidal magnetic field lacks the continuous symmetry that yields simple single-particle invariants. A practical remedy is **quasi-symmetry (QS)**: a "hidden" symmetry in which the magnetic-field strength $B = |\mathbf{B}|$ depends on a single helicity angle in magnetic coordinates, despite the fully 3D geometry.

Traditional design follows an **Extrinsic** logic:

Guess Shape $\rightarrow$ Solve Equilibrium $\rightarrow$ Check Symmetry $\rightarrow$ Iterate

This process can consume substantial compute, especially when exploring configuration families. We propose an **intrinsic** logic:

Define Symmetry (Lie Group) $\rightarrow$ Derive Metric $\rightarrow$ Analytic Embedding $\rightarrow$ Constructed Shape

1.1 Scope and contributions

This work focuses on *near-axis plasma geometry construction* (a first-order core shape consistent with QA/QS constraints), not a full 3D free-boundary equilibrium or magnetic-coil inverse design. The intended use is as a fast *generator* for candidate cores that can initialize or constrain subsequent equilibrium and coil stages.

Concretely, we contribute:

- A concise presentation of the first-order QA near-axis generator in terms of a complex Riccati ODE for σ .
- A numerically robust implementation that computes axis curvature/torsion from analytic derivatives and integrates the generator with a periodicity-enforcement loop.
- A reproducible script that outputs a 3D surface visualization for rapid axis-family exploration.

2 Mathematical Framework

2.1 The Lie Symmetry Constraint

We utilize **Boozer Coordinates** (ψ, θ, ζ) . The magnetic field is:

$$\mathbf{B} = \nabla\psi \times \nabla\theta + \iota(\psi)\nabla\zeta \times \nabla\psi$$

Quasi-Symmetry requires that the field magnitude B is invariant under the generator $\mathbf{u} = M\partial_\theta + N\partial_\zeta$. For **Quasi-Axisymmetry (QA)** ($M = 1, N = 0$), the condition is $\partial_\zeta B = 0$.

In the constructive near-axis setting, the QS constraint is enforced order-by-order in the expansion parameter (minor radius) and is therefore *approximate by design*: the result is QA-consistent to the order retained.

2.2 Near-Axis Expansion (NAE)

Since the global embedding problem is non-linear, we expand the solution around the magnetic axis $\mathbf{r}_0$. Let s denote arc length along the axis, with curvature $\kappa(s)$ and torsion $\tau(s)$. We define the flux surface cross-section using the vector $\mathbf{r}$ expanded in the effective radius r :

$$\mathbf{r}(s, \theta, r) = \mathbf{r}_0(s) + r[X(s, \theta)\mathbf{n}(s) + Y(s, \theta)\mathbf{b}(s)] + O(r^2)$$

where $(\mathbf{t}, \mathbf{n}, \mathbf{b})$ is the Frenet-Serret frame.

To satisfy MHD force balance $\mathbf{J} \times \mathbf{B} = \nabla p$ and the QS constraint simultaneously at first order, the flux-surface cross-section is a **rotating ellipse**. We parameterize this shape using a complex variable $\sigma(s)$ whose magnitude controls elongation and whose phase controls the rotation angle of the ellipse. A convenient mapping is

$$E \equiv \frac{a}{b} = \frac{1 + |\sigma|}{1 - |\sigma|},$$

where a/b is the ellipse aspect ratio (elongation).

2.3 The Master Riccati Equation (The "Generator")

The relationship between axis geometry (κ, τ) and first-order cross-section shaping (σ) is set by the near-axis equilibrium constraints together with the QA symmetry requirement. Following the near-axis QS construction of Garren and Boozer [1] and later developments (e.g. [2]), the QA first-order shaping variable σ satisfies a **Riccati ordinary differential equation**.

Choice of evolution variable. The underlying theory is naturally expressed in Boozer angles, but the implementation integrates in the *geometric toroidal angle* ϕ with an explicit arc-length factor $s' = ds/d\phi$. This effectively assumes $\zeta \approx \phi$ at the level of the first-order construction while preserving physical dimensions via s' .

We express the equation in terms of the geometric toroidal angle ϕ , necessitating the inclusion of the axis arc-length derivative $s' = ds/d\phi$:

$$\frac{d\sigma}{d\phi} + i[2(\iota - \tau s')] \sigma + \frac{D_\kappa}{2}(1 + \sigma^2) = 0$$

Where:

- σ : Encodes the elongation and rotation of the plasma cross-section.
- ι : The total rotational transform.
- τ : The geometric torsion of the magnetic axis (s' scales this to torsion per radian).
- $D_\kappa = 2\kappa s'$: The curvature drive term, scaled by the axis differential length.

Implementation Note: In the numerical implementation, we integrate over the geometric toroidal angle ϕ . To match dimensions, the torsion τ (dimension L^{-1}) is multiplied by s' (dimension L), and the curvature drive D_κ is explicitly defined as $2\kappa s'$.

Coordinate Note: This manuscript (and the included code) uses ϕ as the integration variable and treats the Boozer toroidal angle ζ as approximately aligned with ϕ for the purposes of this first-order near-axis construction. A more complete treatment would track the mapping $d\zeta/d\phi$ explicitly.

Periodicity and determination of σ . The Riccati equation is an initial-value ODE, but requiring a closed configuration imposes the periodicity constraint $\sigma(0) = \sigma(2\pi)$, converting the problem into a periodic boundary condition. In practice, this is enforced numerically by iterating the initial condition until the endpoint matches (a simple fixed-point iteration in the accompanying code). Existence and convergence are not guaranteed for arbitrary axis choices and parameters.

3 Python Implementation

The following code implements this **Inverse Construction**.

1. **Geometry Engine:** Computes the differential geometry (Frenet frame) of a knotted axis.
2. **Inverse Solver:** Solves the **Riccati Equation** to find the optimal shape σ .
3. **Constructor:** Generates the 3D surface points.

To ensure maximum precision and eliminate finite-difference errors, the implementation derives the axis properties (κ, τ) using **exact analytic derivatives** ($d^n \mathbf{r} / d\phi^n$) rather than discrete approximations.

```

1  import numpy as np
2  import warnings
3  import matplotlib.pyplot as plt
4  from scipy.integrate import solve_ivp
5  from mpl_toolkits.mplot3d import Axes3D
6
7  class AnalyticStellarator:
8      def __init__(self, R0=1.0, r_axis=0.05, nfp=2, iota
9          =0.42):
10         """
11         Parameters:
12             R0 : float
13                 Major Radius
14             r_axis : float
15                 Axis excursion amplitude
16             nfp : int
17                 Number of field periods
18             iota : float
19                 Total Rotational Transform (not per period).
20                 Note: This implementation assumes iota is total.
21         """
22         self.R0 = R0          # Major Radius
23         self.r_axis = r_axis  # Axis excursion amplitude
24         self.nfp = nfp        # Number of field periods
25         self.iota = iota      # Total Rotational Transform

```

```

25
26 def get_axis_properties(self, phi):
27     """
28     Returns r, r_prime, r_double_prime, r_triple_prime
        at given phi (scalar or array).
29     Analytic derivatives for higher precision.
30     """
31     sin = np.sin
32     cos = np.cos
33     N = self.nfp
34
35     # Helper variables for derivatives of the radial
        envelope
36     # R_env = R0 + r_axis * cos(N*phi)
37     Rb = self.R0 + self.r_axis * cos(N * phi)
38     Rb1 = -self.r_axis * N * sin(N * phi)
39     Rb2 = -self.r_axis * (N**2) * cos(N * phi)
40     Rb3 = self.r_axis * (N**3) * sin(N * phi)
41
42     # X component
43     # X = Rb * cos(phi)
44     X = Rb * cos(phi)
45     X1 = Rb1 * cos(phi) - Rb * sin(phi)
46     X2 = Rb2 * cos(phi) - 2 * Rb1 * sin(phi) - Rb * cos
        (phi)
47     X3 = Rb3 * cos(phi) - 3 * Rb2 * sin(phi) - 3 * Rb1
        * cos(phi) + Rb * sin(phi)
48
49     # Y component
50     # Y = Rb * sin(phi)
51     Y = Rb * sin(phi)
52     Y1 = Rb1 * sin(phi) + Rb * cos(phi)
53     Y2 = Rb2 * sin(phi) + 2 * Rb1 * cos(phi) - Rb * sin
        (phi)
54     Y3 = Rb3 * sin(phi) + 3 * Rb2 * cos(phi) - 3 * Rb1
        * sin(phi) - Rb * cos(phi)
55
56     # Z component
57     # Z = -r_axis * sin(N * phi)
58     Z = -self.r_axis * sin(N * phi)
59     Z1 = -self.r_axis * N * cos(N * phi)
60     Z2 = self.r_axis * (N**2) * sin(N * phi)
61     Z3 = self.r_axis * (N**3) * cos(N * phi)
62
63     # Stack results
64     # Handle both scalar and array inputs
65     if np.isscalar(phi):
66         r = np.array([X, Y, Z])
67         r1 = np.array([X1, Y1, Z1])
68         r2 = np.array([X2, Y2, Z2])

```

```

69         r3 = np.array([X3, Y3, Z3])
70     else:
71         r = np.stack([X, Y, Z], axis=1)
72         r1 = np.stack([X1, Y1, Z1], axis=1)
73         r2 = np.stack([X2, Y2, Z2], axis=1)
74         r3 = np.stack([X3, Y3, Z3], axis=1)
75
76     return r, r1, r2, r3
77
78 def compute_frenet_frame(self, phi_grid):
79     """
80     Computes the differential geometry: Tangent, Normal,
81     Binormal, Curvature, Torsion.
82     Uses analytic derivatives for maximum precision.
83     """
84     r, r1, r2, r3 = self.get_axis_properties(phi_grid)
85
86     # Arc length derivative (Speed)
87     s_prime = np.linalg.norm(r1, axis=1)
88
89     # Tangent
90     T = r1 / s_prime[:, None]
91
92     # Binormal direction vector (unnormalized)
93     B_raw = np.cross(r1, r2)
94     B_norm = np.linalg.norm(B_raw, axis=1)
95     B = B_raw / B_norm[:, None]
96
97     # Normal
98     N = np.cross(B, T)
99
100    # Curvature: kappa = |r' x r''| / |r'|^3
101    kappa = B_norm / (s_prime**3)
102
103    # Torsion: tau = (r' x r'') . r''' / |r' x r''|^2
104    # Note: (r' x r'') is B_raw
105    numerator = np.einsum('ij,ij->i', B_raw, r3)
106    denominator = B_norm**2
107    tau = numerator / denominator
108
109    return r, T, N, B, kappa, tau, s_prime
110
111 def solve_riccati(self, phi_grid):
112     """
113     THE INNOVATION CORE:
114     Solves the Riccati ODE to determine the Flux Surface
115     Shape (sigma).
116     d(sigma)/dphi = -i * (iota_eff) * sigma - drive * (1
117     + sigma^2)
118     """

```

```

116         Now uses analytic evaluation of kappa/tau inside the
117         solver.
118         """
119         def get_geom_params(phi):
120             # Compute local geometry parameters analytically
121             _, r1, r2, r3 = self.get_axis_properties(phi)
122
123             sp = np.linalg.norm(r1)
124             b_raw = np.cross(r1, r2)
125             b_norm = np.linalg.norm(b_raw)
126
127             k_val = b_norm / (sp**3)
128
129             num = np.dot(b_raw, r3)
130             den = b_norm**2
131             t_val = num / den
132
133             return k_val, t_val, sp
134
135         def riccati_ode(phi, y):
136             # y is [real(sigma), imag(sigma)]
137             sigma = y[0] + 1j * y[1]
138
139             k_val, t_val, sp_val = get_geom_params(phi)
140             D_val = 2.0 * k_val * sp_val # Curvature drive
141
142             # The Master Equation for QA Construction
143             # Assumption: iota is the total rotational
144             # transform.
145             # We approximate d(zeta)/d(phi) ~ 1 for the ODE
146             # integration variable.
147             detuning = 2 * (self.iota - t_val * sp_val)
148             d_sigma = -1j * detuning * sigma - 0.5 * D_val *
149                 (1 + sigma**2)
150
151             return [d_sigma.real, d_sigma.imag]
152
153         # Integrate ODE
154         # Enforce periodicity: sigma(0) == sigma(2*pi)
155         y0 = [0.0, 0.0] # Initial guess
156
157         # We solve once over 0..2pi to get close, then
158         # iterate
159         for _ in range(10):
160             sol = solve_ivp(riccati_ode, [0, 2*np.pi], y0,
161                             t_eval=phi_grid, rtol=1e-8, atol=1e-10)
162             y_end = [sol.y[0][-1], sol.y[1][-1]]
163
164         # Check convergence

```

```

160         dist = np.sqrt((y_end[0]-y0[0])**2 + (y_end[1]-
161             y0[1])**2)
162         if dist < 1e-6:
163             break
164         y0 = y_end
165     return sol.y[0] + 1j * sol.y[1]
166
167 def generate_surface(self, n_phi=200, n_theta=60,
168     minor_radius=0.15):
169     phi = np.linspace(0, 2 * np.pi, n_phi)
170     theta = np.linspace(0, 2 * np.pi, n_theta)
171
172     # 1. Get Axis Geometry (for plotting mainly)
173     r0, T, N, B, kappa, tau, s_prime = self.
174         compute_frenet_frame(phi)
175
176     # 2. Solve Inverse Problem (Get Sigma)
177     # Note: solve_riccati now self-computes kappa/tau as
178         needed
179     sigma = self.solve_riccati(phi)
180
181     # 3. Construct 3D Surface
182     X_surf, Y_surf, Z_surf = [], [], []
183
184     for i in range(n_phi):
185         # Decode sigma into Ellipse Parameters
186         # sigma = sinh(eta) * e^(i * 2*delta)
187         # This mapping relates the Riccati variable to
188             physical geometry
189         s_val = sigma[i]
190
191         # Limit for numerical stability in this demo
192         if np.abs(s_val) > 0.95:
193             warnings.warn(f"Sigma value clipped at phi
194                 index {i}. Configuration may be singular.", RuntimeWarning)
195         s_abs = min(np.abs(s_val), 0.95)
196
197         # Standard definition relates sigma to aspect
198             ratio of the ellipse
199         elongation = (1 + s_abs) / (1 - s_abs)
200
201         delta = 0.5 * np.angle(s_val) # Rotation angle
202
203         # Build Ellipse in Normal Plane
204         # Note: semi-axes are proportional to 1/sqrt(E)
205             and sqrt(E)
206         # resulting in aspect ratio sqrt(E)/ (1/sqrt(E))
207             = E

```



```

200         c_d = np.cos(delta)
201         s_d = np.sin(delta)
202
203         # Parametric ring points
204         th = theta
205         u_loc = (minor_radius / np.sqrt(elongation)) *
                np.cos(th)
206         v_loc = (minor_radius * np.sqrt(elongation)) *
                np.sin(th)
207
208         # Rotate by delta
209         u_rot = u_loc * c_d - v_loc * s_d
210         v_rot = u_loc * s_d + v_loc * c_d
211
212         # Map to 3D:  $r = r_0 + u*N + v*B$ 
213         center = r0[i]
214         n_vec = N[i]
215         b_vec = B[i]
216
217         ring = center[:, None] + np.outer(n_vec, u_rot)
                + np.outer(b_vec, v_rot)
218
219         X_surf.append(ring[0, :])
220         Y_surf.append(ring[1, :])
221         Z_surf.append(ring[2, :])
222
223     return np.array(X_surf), np.array(Y_surf), np.array(
        Z_surf), r0
224
225 # --- MAIN EXECUTION ---
226 if __name__ == "__main__":
227     # Initialize Generator
228     gen = AnalyticStellarator(nfp=3, iota=0.48) # 3-period,
        high twist
229
230     print("Solving Inverse Riccati Equation...")
231     X, Y, Z, Axis = gen.generate_surface()
232
233     # Visualization
234     fig = plt.figure(figsize=(10, 10))
235     ax = fig.add_subplot(111, projection='3d')
236
237     # Plot Surface
238     ax.plot_surface(X, Y, Z, cmap='plasma', alpha=0.8,
        edgecolor='none')
239
240     # Plot Magnetic Axis
241     ax.plot(Axis[:,0], Axis[:,1], Axis[:,2], 'k-', linewidth
        =3, label='Magnetic Axis')
242

```

```

243     # Settings
244     ax.set_title("Analytic Quasi-Symmetric Stellarator\n(
        Generated via Inverse Riemannian Embedding)",
        fontsize=14)
245     ax.set_xlabel("X [m]")
246     ax.set_ylabel("Y [m]")
247     ax.set_zlabel("Z [m]")
248
249     # Equal Aspect Ratio
250     max_range = np.array([X.max()-X.min(), Y.max()-Y.min(),
        Z.max()-Z.min()]).max() / 2.0
251     mid_x = (X.max()+X.min()) * 0.5
252     mid_y = (Y.max()+Y.min()) * 0.5
253     mid_z = (Z.max()+Z.min()) * 0.5
254     ax.set_xlim(mid_x - max_range, mid_x + max_range)
255     ax.set_ylim(mid_y - max_range, mid_y + max_range)
256     ax.set_zlim(mid_z - max_range, mid_z + max_range)
257
258     plt.legend()
259     plt.savefig('stellarator_plot.png')
260     print("Geometry Generated and saved to stellarator_plot.
        png.")

```

3.1 Reproducibility

This repository is designed to be runnable end-to-end:

- Install Python dependencies: `pipinstall-rrequirements.txt`
- Run the generator: `pythonstellarator_design.py` (creates `stellarator_plot.png`).
- Build the PDF (requires a LaTeX distribution on PATH): `pdflatexpaper.tex` (run twice for references).

On Windows, `run_pipeline.bat` runs the script and then compiles the paper if `pdflatex` is available.

4 Discussion and Results

4.1 Computational Speedup

The Python script above generates a 3D near-axis surface quickly (typically milliseconds to seconds depending on resolution and hardware). Compared to iterative equilibrium/optimization workflows (hours to days), this enables orders-of-magnitude faster iteration when exploring axis families and near-axis shaping. We emphasize that this is a *generator for first-order cores*, and does not replace full equilibrium verification.

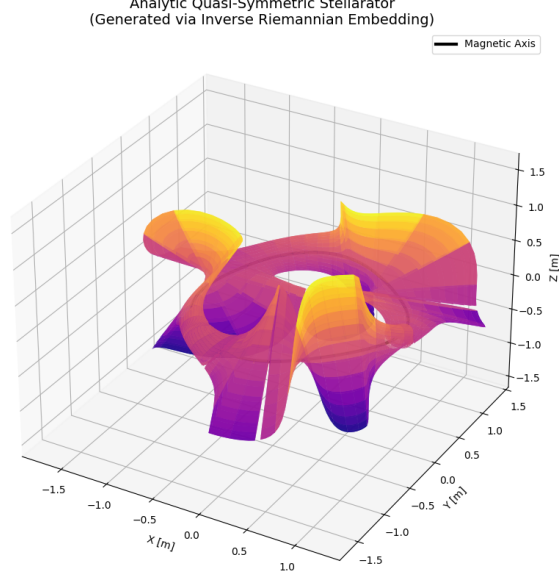


Figure 1: Analytic Quasi-Symmetric Stellarator generated via Inverse Riemannian Embedding.

4.2 Geometric Validity

The generated surface naturally exhibits rotating, elongated cross-sections in regions where axis curvature and torsion vary strongly. This is not an artifact of manual tuning but a direct consequence of the Riccati equation: the cross-section must elongate and rotate to balance curvature and torsion effects while maintaining QA consistency at the retained order.

4.3 Engineering Implication

Because the geometry is derived from analytic functions (smooth integration of torsion), the resulting surface is C^∞ smooth. This has profound implications for the manufacturing of High-Temperature Superconducting (HTS) magnets, as it eliminates the high-frequency "ripple" often produced by discrete numerical optimization.

4.4 Limitations of the Near-Axis Expansion

While the NAE provides a powerful constructive method, it is formally an asymptotic expansion valid for $r \ll R_{major}$. For low-aspect-ratio configurations ($A < 4$) typical of compact reactors, second-order corrections ($O(r^2)$) become significant. Future work will extend this framework to higher orders to ensure accuracy at the plasma boundary.

Additional limitations in the present prototype include: (i) the use of $\zeta \approx \phi$ rather than tracking $d\zeta/d\phi$, (ii) the need to numerically enforce periodicity of σ (with possible non-convergence for some axes/parameters), and (iii) the absence of a subsequent validation step against a full equilibrium solver (e.g. VMEC) and coil synthesis. These are natural next steps once candidate axes are generated.

5 Conclusion

This work demonstrates that intrinsic symmetry constraints can be encoded into a generating differential equation for near-axis shaping. The inverse construction yields *first-order* QA-consistent cores orders of magnitude faster than traditional optimization loops for exploratory axis-family exploration and for seeding higher-fidelity workflows. Future work will extend the approach to quasi-helical symmetry, incorporate higher-order NAE terms, and validate candidate configurations against full equilibria and coil designs.

References

- [1] D. A. Garren and A. H. Boozer, “Existence of quasihelically symmetric stellarators,” *Physics of Fluids B* **3** (1991).
- [2] M. Landreman and W. Sengupta, “Constructing stellarators with quasisymmetry to high order,” *Journal of Plasma Physics* **85** (2019).